



PETER THE GREAT ST PETERSBURG POLYTECHNIC UNIVERSITY
Institute of Cybersecurity and Computer Science

PRACTICAL TASK REPORT

Fuzzy Logic Modeling in Python

Stress Regulation Feedback Controller

Task 2

Student Mohammad Azimi

Group 5140901/61701

Course Intelligent Systems

Instructor Yuri Nurgalievich Kozhubaev

Saint Petersburg
2026

1 Introduction

This report documents a Mamdani fuzzy controller that converts two input scores, stress and physiological arousal, into a feedback intensity from 0 to 100 percent. The controller was implemented in Python and tested in Visual Studio Code using the scikit-fuzzy workflow presented in the course study guide [1]. The baseline system produced interpretable low, moderate, and strong feedback levels, while a modified parameter set increased sensitivity in transition regions by as much as 13.43 percentage points.

Fuzzy logic represents gradual membership instead of imposing sharp boundaries between linguistic states such as low, medium, and high [3]. This property is useful when inputs are uncertain or subjective. In this educational model, the input values are entered manually and do not represent a clinically validated diagnosis.

1.1 Objective

The objective was to create and study a complete fuzzy inference system in Python, verify its behavior across representative input combinations, modify its membership-function parameters, and measure how those changes affect the output.

- Define fuzzy input and output variables with overlapping membership functions.
- Construct a nine-rule Mamdani rule base.
- Run an interactive simulation and visualize the crisp input and output values.
- Evaluate all nine representative low, medium, and high combinations.
- Shift the input membership functions and compare the baseline and modified controllers.

1.2 Software Environment

Component	Version or role
Operating system	Windows 10
Development environment	Visual Studio Code
Python	3.12.6
scikit-fuzzy	0.5.0
NumPy	2.5.3
SciPy	1.18.1
Matplotlib	3.11.2

2 Fuzzy Controller Design

The controller uses two antecedents and one consequent. Both input universes range from 0 to 10 with a resolution of 0.1. The output universe ranges from 0 to 100 percent with a resolution of one percentage point.

Variable	Role	Range	Linguistic terms
Stress	Input	0 to 10	Low, Medium, High
Arousal	Input	0 to 10	Low, Medium, High
Feedback intensity	Output	0 to 100 percent	Gentle, Moderate, Strong

2.1 Baseline Membership Parameters

Trapezoidal functions describe the outer terms because low and high values should remain fully active near the limits. Triangular functions describe the central terms. Overlap between adjacent sets allows a single crisp input to activate more than one rule [1, 2].

Variable	Term	Function	Parameters
Stress and arousal	Low	Trapezoidal	[0, 0, 2, 4]
Stress and arousal	Medium	Triangular	[2, 5, 8]
Stress and arousal	High	Trapezoidal	[6, 8, 10, 10]
Feedback intensity	Gentle	Trapezoidal	[0, 0, 15, 35]
Feedback intensity	Moderate	Triangular	[20, 50, 80]
Feedback intensity	Strong	Trapezoidal	[65, 85, 100, 100]

2.2 Membership Functions

Figure 1 shows the complete baseline membership design. The input curves use the same parameters so that stress and arousal have consistent linguistic interpretations. The output sets overlap from gentle to moderate and from moderate to strong.

Fuzzy Stress Regulation Controller

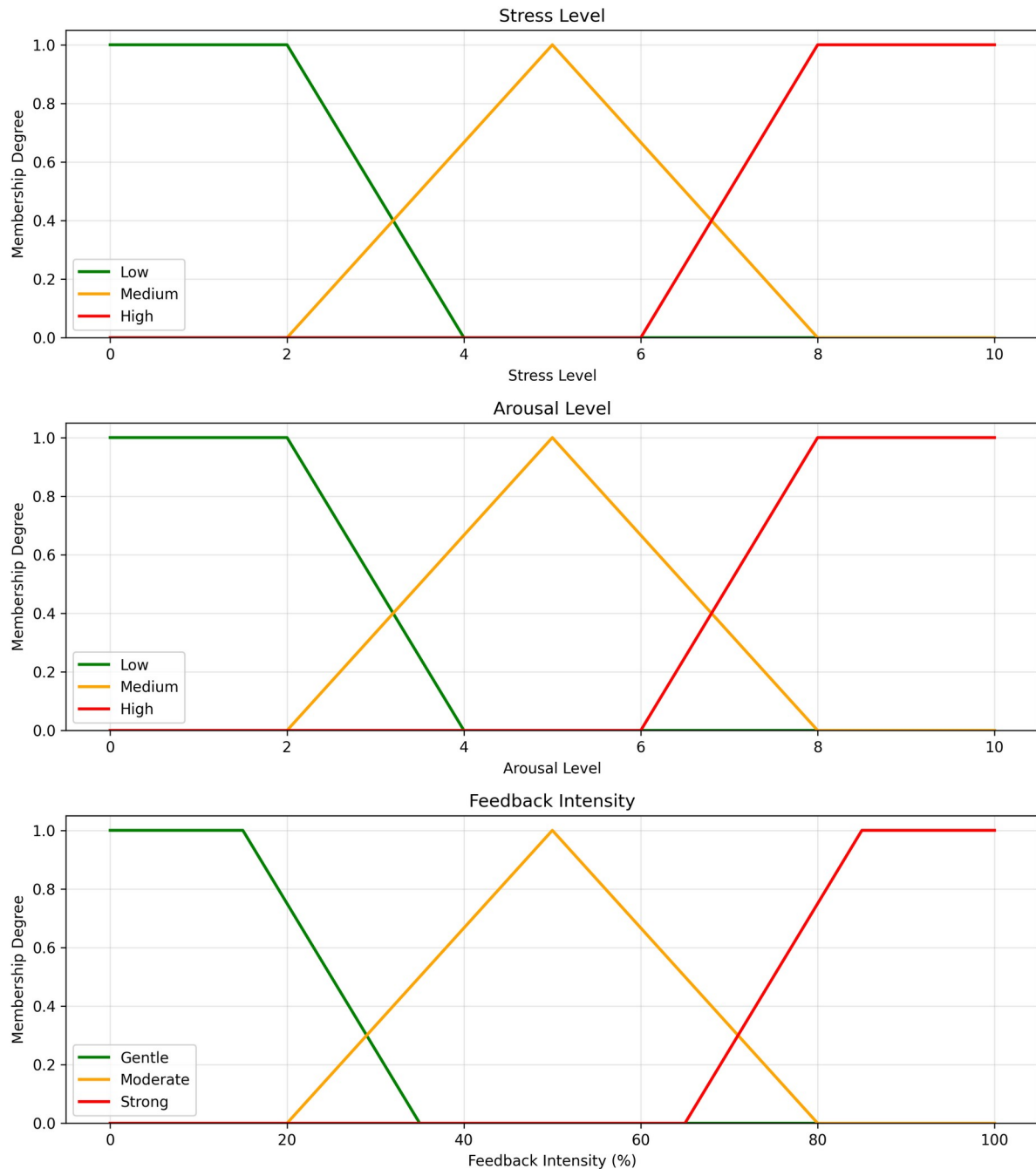


Figure 1 Baseline membership functions for stress arousal and feedback intensity

3 Rule Base and Inference

The rule base contains every combination of the three stress levels and three arousal levels. Higher values in either input increase the recommended feedback intensity. The complete mapping is shown in Table 3.

Stress and arousal	Low arousal	Medium arousal	High arousal
Low stress	Gentle	Gentle	Moderate
Medium stress	Gentle	Moderate	Strong
High stress	Moderate	Strong	Strong

3.1 Inference Sequence

- Fuzzification converts each crisp input into membership degrees.
- The AND operator uses the minimum membership degree to determine rule activation.
- Rule outputs are aggregated with the maximum operator.
- Centroid defuzzification converts the aggregated output into one percentage value [1, 4, 5].

Listing 1 Core rule and simulation operations

```
rule = ctrl.Rule(stress["high"] & arousal["medium"],
                 feedback_intensity["strong"])
control_system = ctrl.ControlSystem(rules)
simulation = ctrl.ControlSystemSimulation(control_system)
simulation.input["stress"] = stress_value
simulation.input["arousal"] = arousal_value
simulation.compute()
result = simulation.output["feedback_intensity"]
```

The implementation validates that both input values remain between 0 and 10. It then prints the crisp output, identifies the output term with the greatest membership degree, suggests an illustrative feedback action, and saves a result plot.

4 Program Operation

The interactive demonstration used a stress value of 8 and an arousal value of 7. The fuzzy system returned 84.86 percent, classified the result as Strong, and suggested combined calming feedback with a rest reminder. The blue dashed lines in Figure 2 mark the crisp inputs and defuzzified output.

Stress	Arousal	Feedback intensity	Category
8.00	7.00	84.86 percent	Strong

Fuzzy Controller Simulation Result

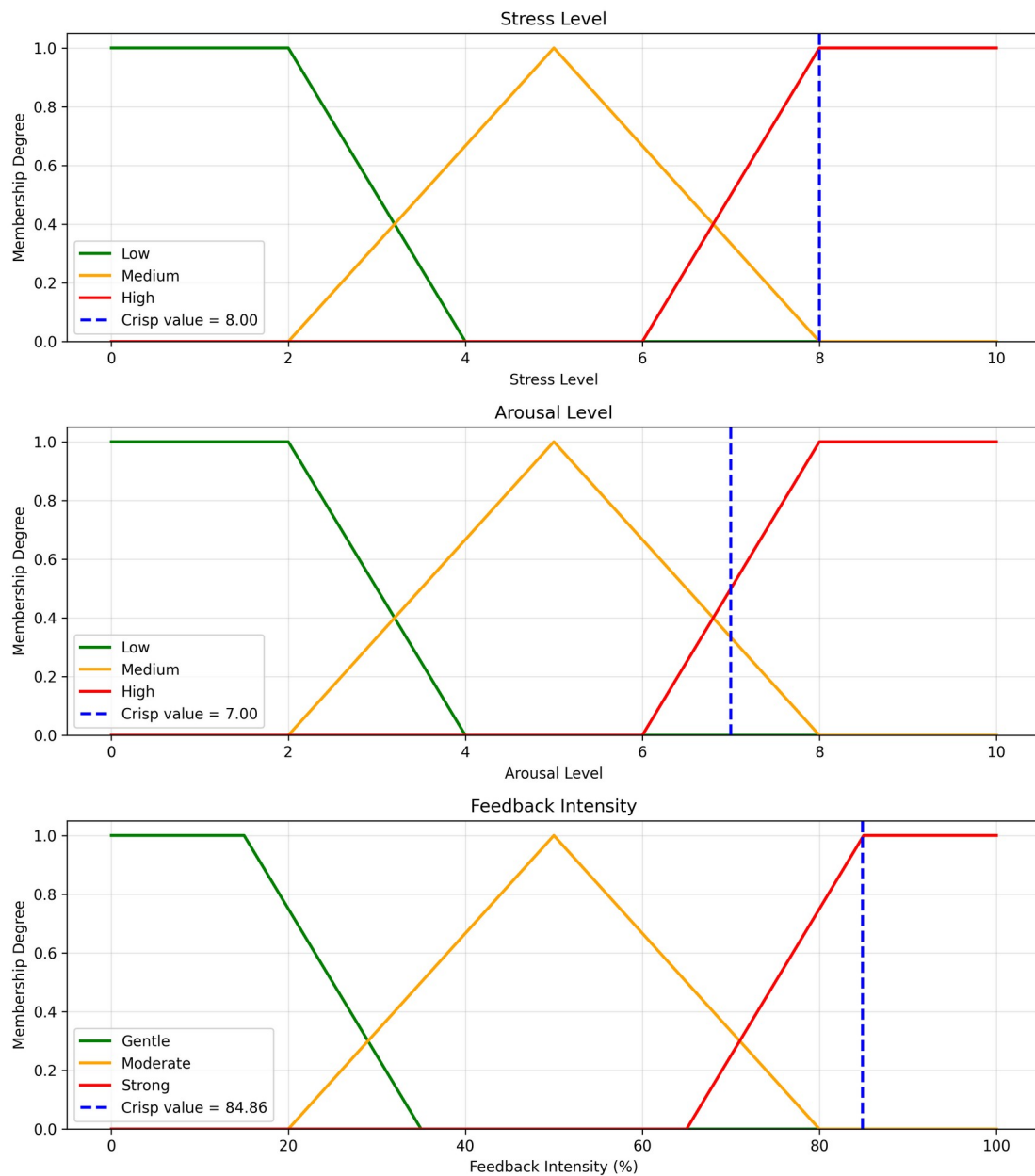


Figure 2 Simulation result for stress 8 and arousal 7

5 Nine Scenario Evaluation

Representative crisp values of 1, 5, and 9 were used for the low, medium, and high states. Testing the resulting nine combinations confirmed that the rule base behaves monotonically: stronger stress or arousal does not reduce the feedback intensity.

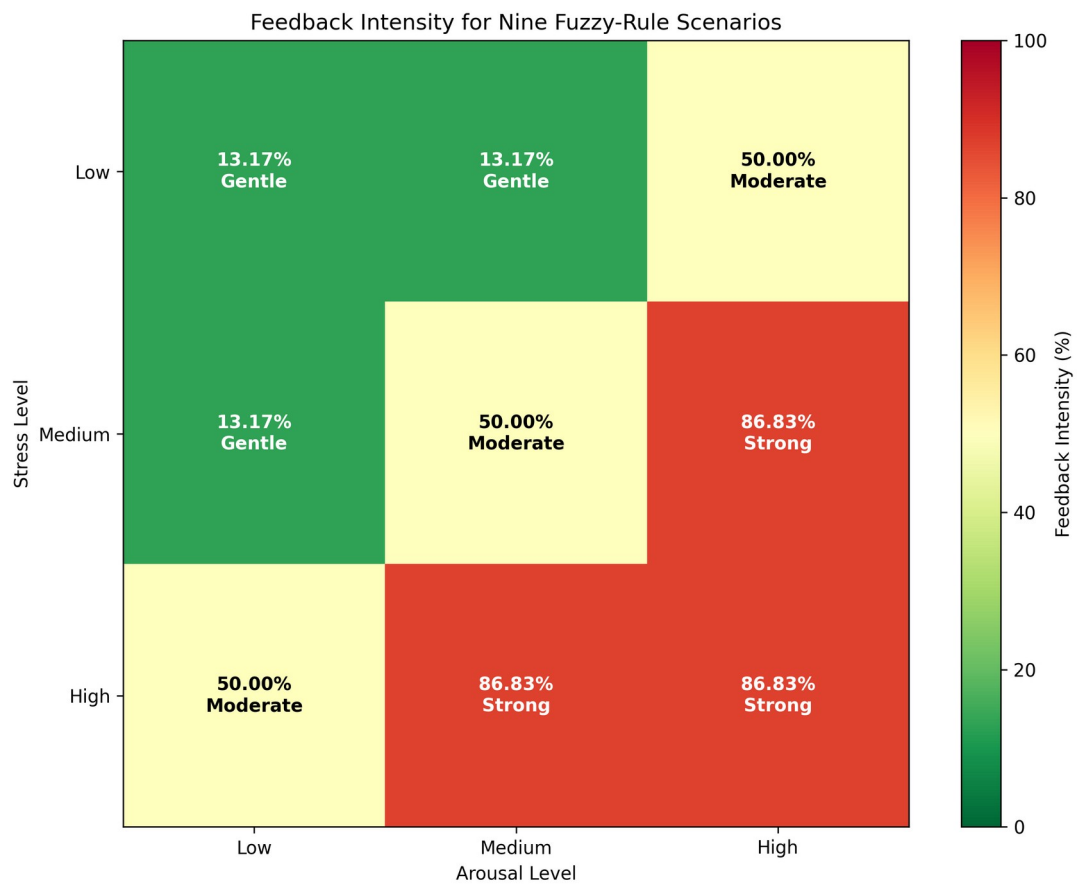


Figure 3 Feedback intensity for the nine representative rule scenarios

Low-low, low-medium, and medium-low combinations produced 13.17 percent and Gentle feedback. Mixed low-high or high-low combinations produced 50.00 percent and Moderate feedback. Medium-high, high-medium, and high-high combinations produced 86.83 percent and Strong feedback.

6 Membership Parameter Modification

The input membership functions were shifted toward lower values to create a more sensitive controller. The rule base and all output membership functions remained unchanged. This isolates the effect of modifying the input parameters.

Input term	Baseline parameters	Modified parameters	Change
Low	[0, 0, 2, 4]	[0, 0, 1.5, 3.5]	Upper boundary moved left
Medium	[2, 5, 8]	[1.5, 4.5, 7.5]	Peak and boundaries moved left
High	[6, 8, 10, 10]	[5, 7, 10, 10]	High state activates earlier

Baseline vs Modified Input Membership Functions

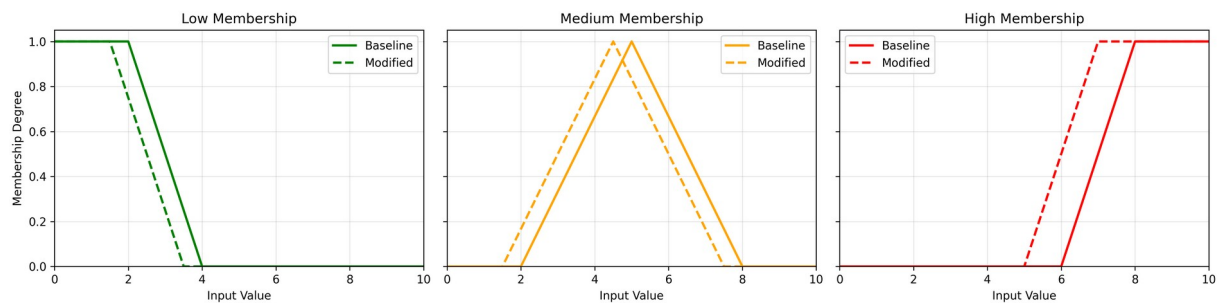


Figure 4 Baseline and modified input membership functions

The dashed curves in Figure 4 show the modified functions. Medium membership now peaks at 4.5 instead of 5, and high membership begins at 5 instead of 6. As a result, transition-region inputs activate stronger rules earlier.

7 Effect of the Parameter Changes

The modified system produced equal or higher feedback for all eight comparison cases. The largest increase was 13.43 percentage points. The central case remained unchanged because both models retain full medium membership near the center.

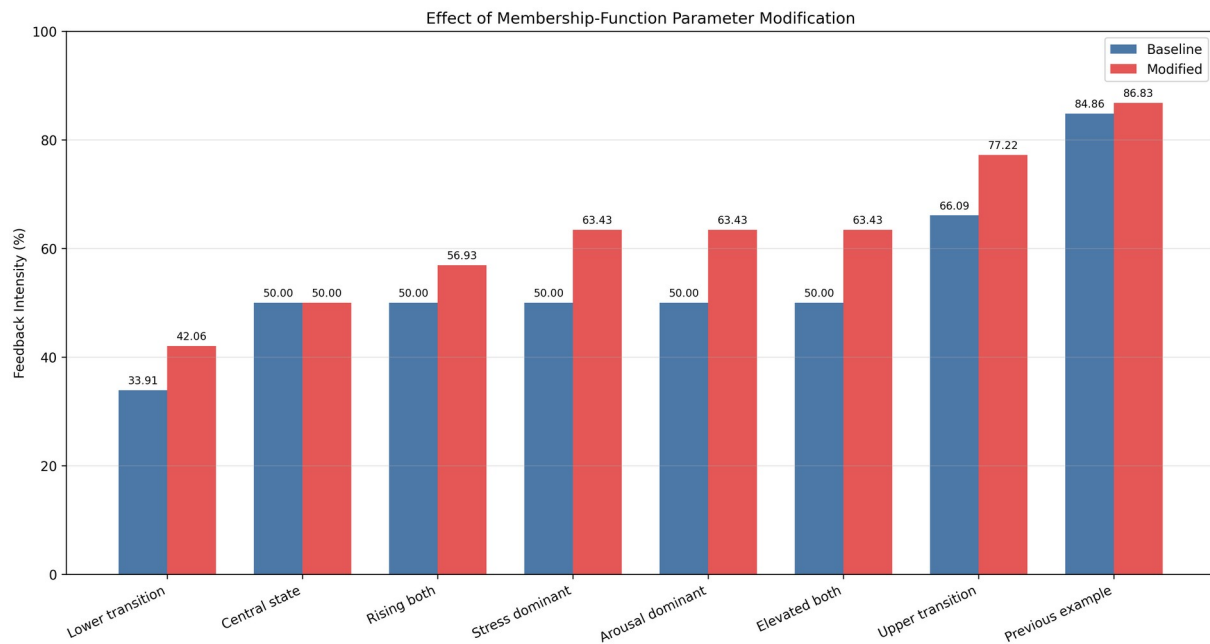


Figure 5 Baseline and modified feedback intensities

Scenario	Baseline	Modified	Difference
Lower transition 3 and 3	33.91	42.06	+8.15
Rising both 5.5 and 5.5	50.00	56.93	+6.93
Elevated both 6 and 6	50.00	63.43	+13.43
Upper transition 7 and 7	66.09	77.22	+11.12
Previous example 8 and 7	84.86	86.83	+1.97

8 Discussion

The baseline controller provides a stable and interpretable mapping from two uncertain inputs to one continuous output. Its overlapping sets avoid abrupt changes at linguistic boundaries. The scenario heatmap confirms that the nine expert rules create the intended ordering from Gentle to Strong feedback.

Changing only the input membership parameters demonstrated the sensitivity of the model. Moving the medium and high sets toward lower values made the controller react earlier in transition regions. The largest differences occurred when one or both inputs were near the boundary between medium and high. The difference became smaller at very high inputs because the baseline system was already near the Strong output plateau.

8.1 Limitations

- Stress and arousal are manually entered scores rather than values estimated from validated physiological sensors.
- The membership parameters and rules are illustrative and were not calibrated with participant data.
- The suggested actions demonstrate controller output only and must not be interpreted as medical advice.
- Clinical use would require validated measurements, safety constraints, personalization, and formal evaluation.

9 Conclusion

The practical task produced a complete fuzzy logic model in Python. The program defines three linguistic sets for each variable, applies nine Mamdani rules, calculates a centroid output, validates user input, and generates report-ready figures and data files. The parameter experiment showed that shifting the input membership functions can increase feedback intensity by up to 13.43 percentage points in the tested transition scenarios. The results confirm that membership-function parameters have a direct and measurable effect on fuzzy controller behavior.

9.1 Running the Project

Listing 2 Commands used to run the project

```
python main.py
python scenario_experiments.py
python parameter_comparison.py
```

References

- [1] Novak, D. A., Kozhubaev, Yu. N., and Ovchinnikova, E. N. Introduction to Information Technologies: Modeling of Fuzzy Control Systems in Python Programming Language. Study guide. Saint Petersburg: AISING, 2024, 81 p.
- [2] Kozhubaev, Yu. N., and Khokhlovsky, V. N. Modelling of Fuzzy Control and Information Processing Systems in the MATLAB/Simulink Software Environment. Study guide. Saint Petersburg: AISING, 2023, 122 p.
- [3] Zadeh, L. A. Fuzzy Sets. *Information and Control*, 8(3), 338-353, 1965.
- [4] Mamdani, E. H., and Assilian, S. An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller. *International Journal of Man-Machine Studies*, 7(1), 1-13, 1975.
- [5] scikit-fuzzy documentation. Fuzzy Control Systems and Control System Simulation. <https://scikit-fuzzy.readthedocs.io/> Accessed 19 September 2026.
- [6] Ross, T. J. *Fuzzy Logic with Engineering Applications*. Third edition. Wiley, 2010.